

CS383 - Software Engineering

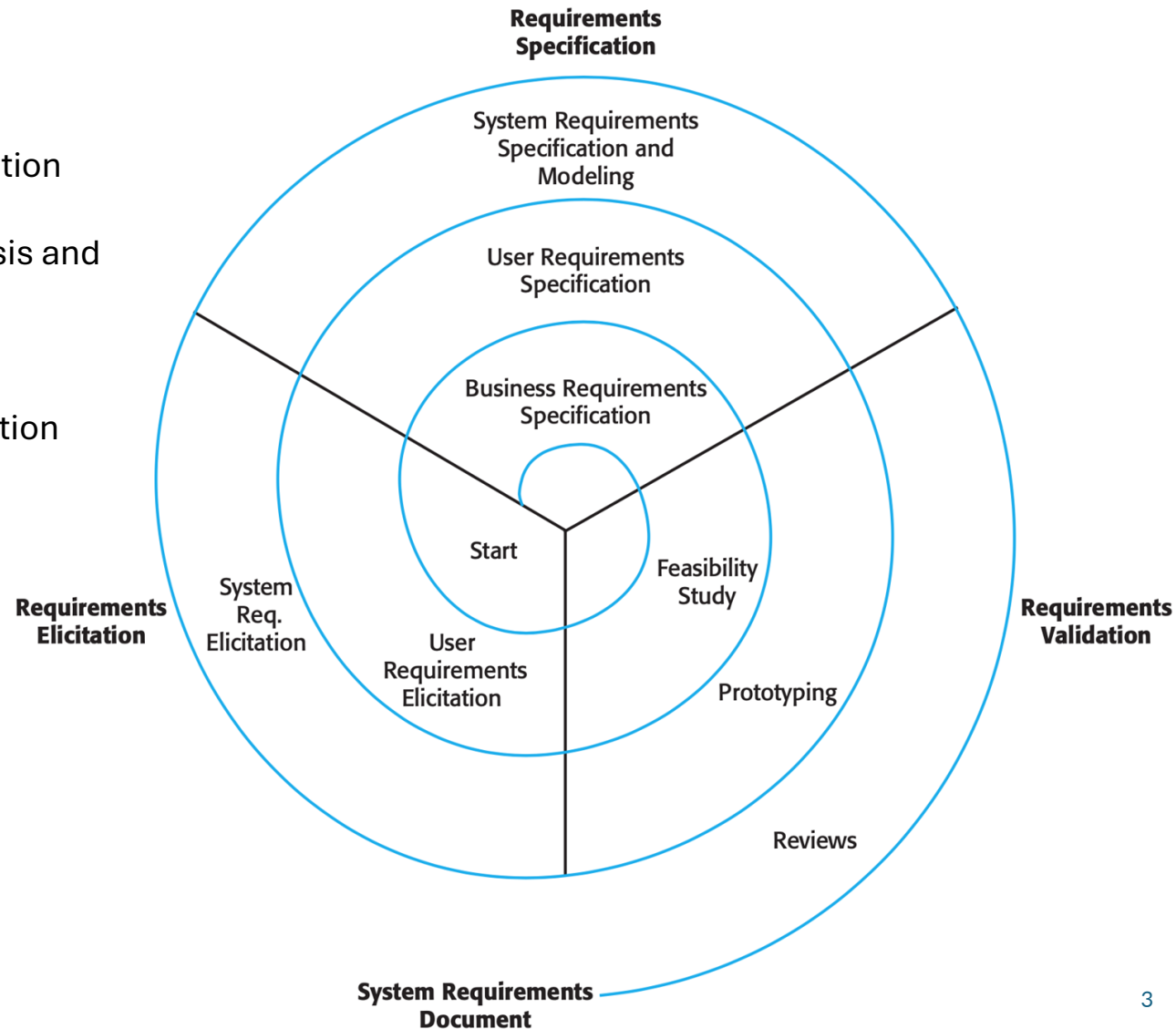
Requirement Engineering

Outline

- Requirement engineering
- Requirements engineering process
- Requirement elicitation
- Requirements analysis
- Requirement validation
- Requirements management

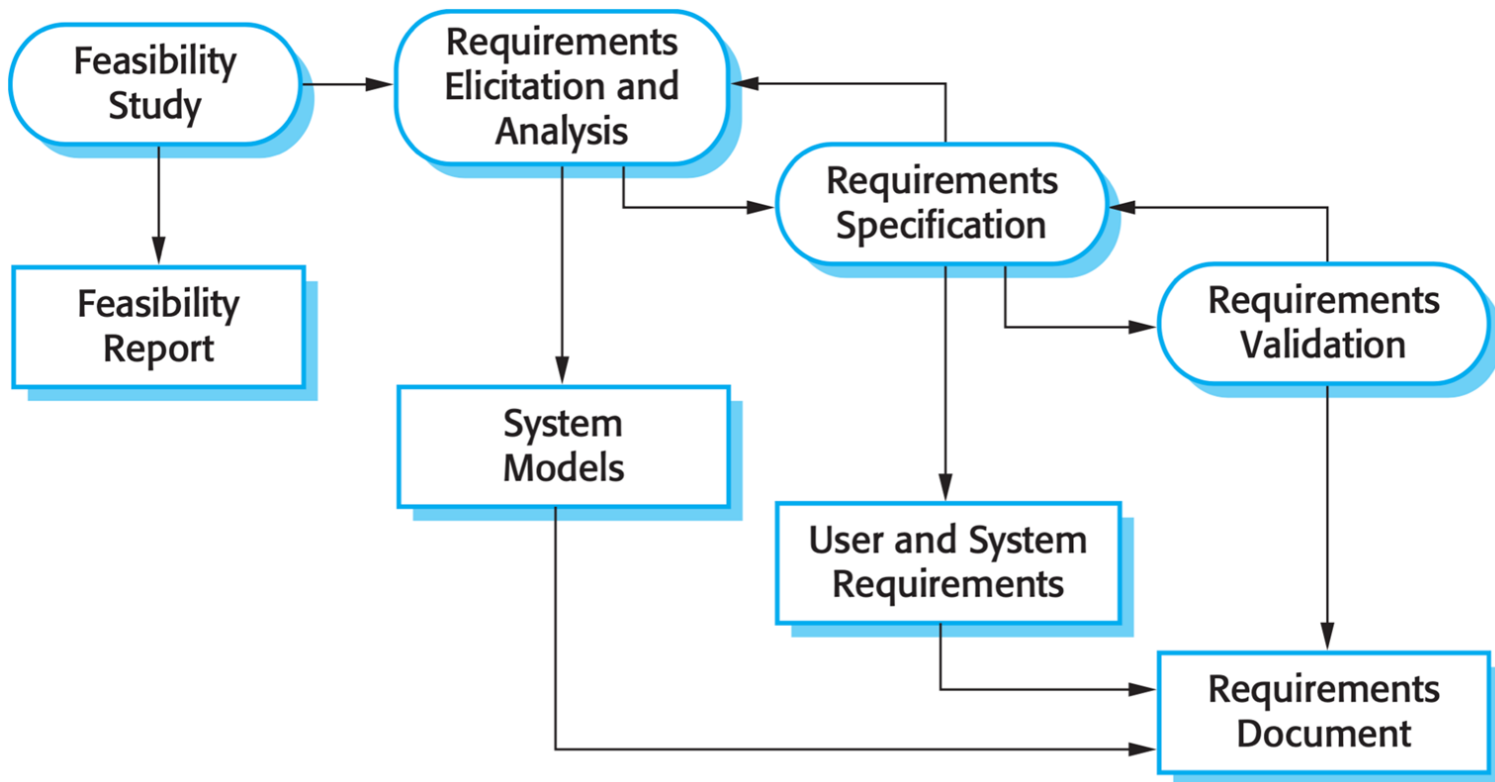
Requirement Engineering Process

1. Requirements elicitation
2. Requirements analysis and specification
3. Requirements validation



Requirement specification activities reminder

- Here is a figure to remind you about the requirement specification from lecture 2.
- Today's lecture goes into detail of the processes:



Feasibility study and what lies ahead

A feasibility study is a short, focused study that should take place early in the RE process. It should answer three key questions:

1. does the system contribute to the overall objectives of the organization and it is economically justifiable?
2. can the system be implemented within schedule and budget using current technology?
3. can the system be integrated with other systems that are used?

If the answer to any of these questions is no, then probably not go ahead with the project.

Requirements elicitation and analysis

Technical staff works with clients to find services that the system should provide. This process is four stages

1. Requirement discovery:

- Interacting with stakeholders of the system to discover their requirements:
interviews, scenarios, use cases, ethnography

2. Requirements classification and organisation

- Take the collection of requirements and organise them into groups
- May use an architectural design to identify system components and then related requirements accordingly to components

Requirements elicitation and analysis (cont.d)

3. Requirements prioritization and negotiation

- Prioritizing requirements and finding and resolving requirements conflicts (different customers/stakeholders have different views)

4. Requirements specification

- Documenting requirements and input into the next round

Requirements elicitation and analysis (cont.d)

Collecting and understanding requirements from system stakeholders can be difficult because of the following reasons:

- They often don't know what they want except in the most general terms
- They express requirements in their own natural terms and based on their knowledge/experience
- Different stakeholders have different requirements and they may express them in different ways
 - Engineers have to discover all requirement sources and discover commonalities and conflict

Requirements elicitation and analysis (cont.d)

- Political factors may influence the requirements.
 - Managers may demand specific requirements that allow them to increase their influence in the organization
- Requirements and their analysis are based on dynamic economic and business environment
 - New requirements may emerge from new stakeholders who were not originally consulted.

Requirements discovery (collection)

- There are techniques for requirements gathering
 - Interviewing
 - Doing formal or informal interview with stakeholders
 - Scenarios
 - Initiate an event of action and build its complete action series
 - Use case
 - Use textual or graphical description to outline the use one particular event (e.g. a one action of a scenarios)
 - Ethnography: a complementary study of social and organizational context of operational processes

Requirements discovery (collection) (cont.d)

The process of gathering information about the required system.

Methods are:

1. Interviewing stakeholders: either formally or informally
 - Ask questions about the required system and its functionalities
 - Also, interviews can be:
 - Closed interviews: stakeholder answers predefined questions
 - Open interviews: no predefined questions but there are issues to be discussed to have better understanding of requirements
 - To be an effective interviewer you must be
 - Open-minded, avoid preconceived ideas about the requirements, willing to listen
 - Prompt interviewee to get discussion going (you need the context to which the requirement will be derived)

Requirements discovery (collection) (cont.d)

2. Scenarios: real-life example of how to interact with target software system

- A scenario starts with an outline interaction and then developed to a complete description of the interaction
- A scenario may include:
 - A description of what to expect from the scenario when it starts
 - A description of the normal flow of events
 - A description of what may go wrong
 - Information about other activities
 - A description of the system state when the scenario finishes.

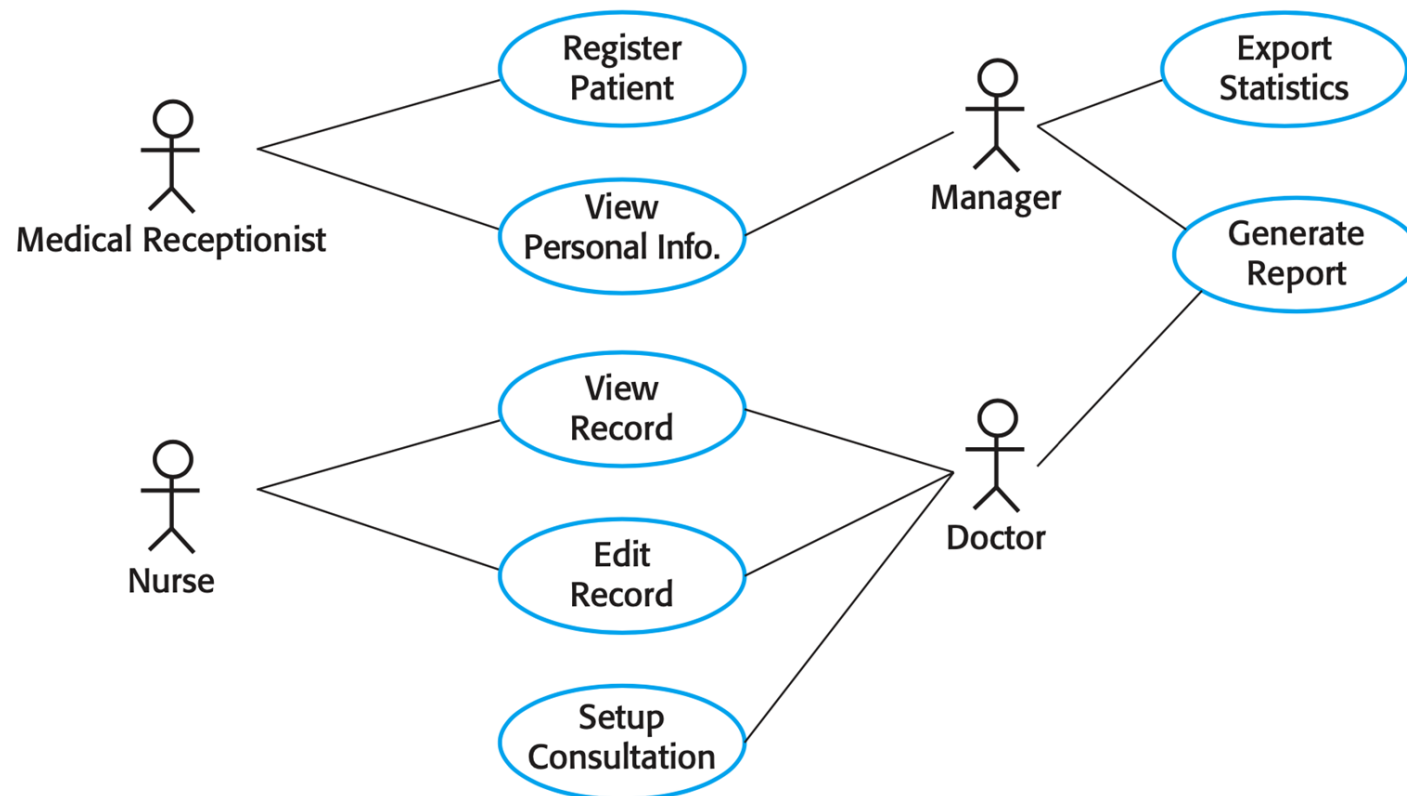
Scenario example

- **Buy a product**
 1. Customer browses through catalogue and selects items to buy
 2. Customer goes to checkout
 3. Customer fills in shipping information if not regular customer; or otherwise goes to step 3.a
 4. System presents full pricing information, including shipping
 5. Customer fills in credit card information
 6. System authorises purchase
 7. System confirms sale immediately
 8. System sends confirmation email to customer
- **Alternative: authorisation failure**
 - 6.a. System fails to authorise credit purchase
 - 6.a.1 Allow customer to re-enter credit card information and retry
- **Alternative: regular customer**
 - 3.a. System displays current shipping information, pricing information, and last four digits of credit card information
 - 3.b. Customer may accept or override these defaults. Return to primary scenario at step 6

Requirements discovery (collection) (cont.d)

3. Use-cases: identifies the actors involved in interactions of a scenario.

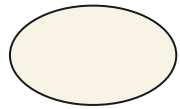
Interaction are documented using diagrams (usually using UML)



Use case

- A method for capturing the functional requirements of a system
 - Describes typical interaction between the actors and the system
 - Clients often don't know what they want (defining functional requirements)
 - Developers don't know the nature of the system (the context and requirements of the system)
 - Provide a narrative of how a system is used
 - Basic flow of events in a use case
 - Can be used to design the user interface
- Use case is a set of actions of interacting with the system

UML Graphical use case



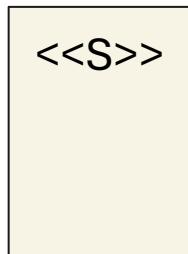
use case



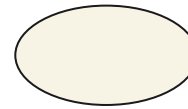
association



actor

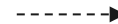


system boundaries



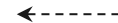
abstract use case

<<I>>

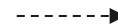


include

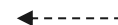
<<E>>



extend



dependency

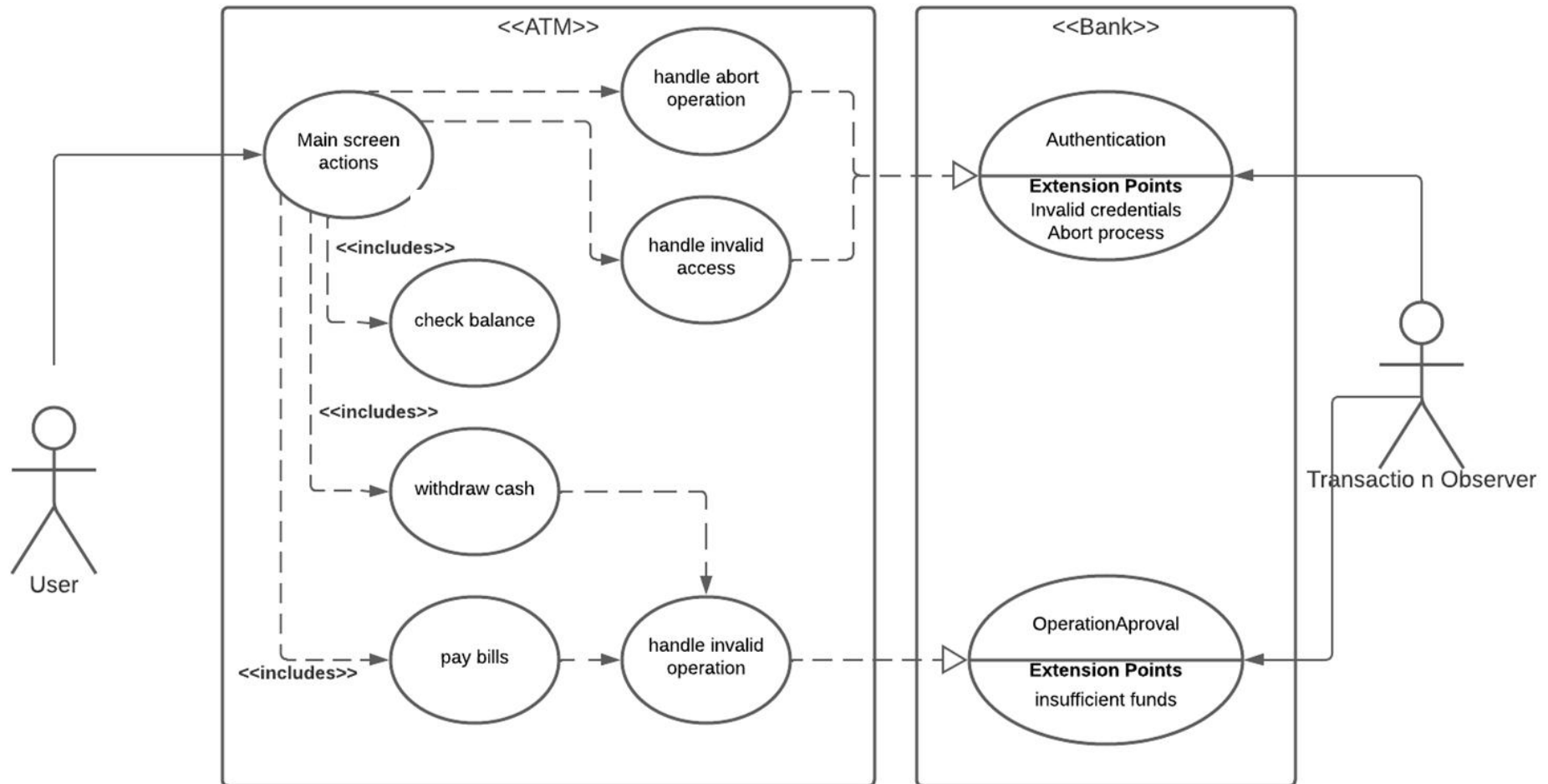


generalization

Use case example 1

A customer accesses a list of bank services via an ATM connected to a bank. The services provided are checking balance, withdrawing cash, paying bills, etc. Doing the operation must be approved by the bank. The ATM handles authentication processes according to bank authentication. There is a person that must observe any transaction (observer).

Use case example 1



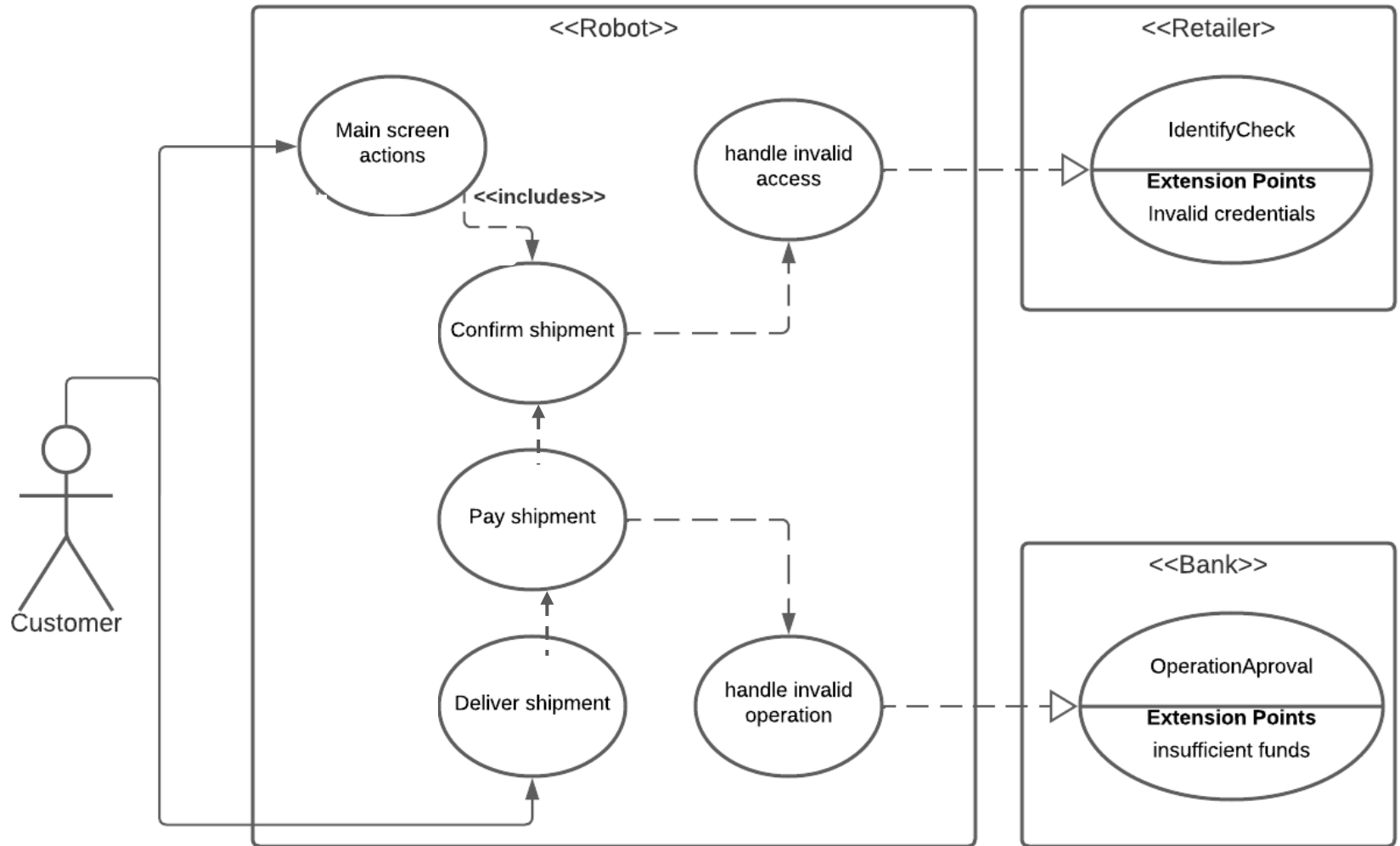
Scenario and use case guidelines

1. Name the system scope and boundaries.
2. Brainstorm and list the primary actors
3. Brainstorm and exhaustively list user goals for the system.
4. Capture the outermost summary use cases to see who really cares.
5. Reconsider and revise the summary use cases: add, subtract, or merge goals.
6. Double-check for time-based triggers and other events at the system boundary.
7. Brainstorm and exhaustively list the extension and conditions.
8. Write the main success scenario (MSS) considering the use cases
9. Write alternative scenarios of main scenario

Use case diagram: exercise 1

An autonomous delivery robot car is a self-driving car that reads surrounding environment and objects and with shipments detail of customers (provided by a retailer), it travels through town and deliver packages. When it arrives to a customer, it either delivers or allow the user to pay for shipment and then deliver the items.

- **Use case:** pay and take
 - Customer wants to access the robot allowed services via the robot screen and pay for the shipment. Once paid, the package(s) is/are delivered. There are two external use cases involved
 - *Identity Authentication of a retailer system*
 - *Bank Approval of a Bank system*
 - **Failures:** occur during at authentication, approving process



Use case diagram: exercise 2

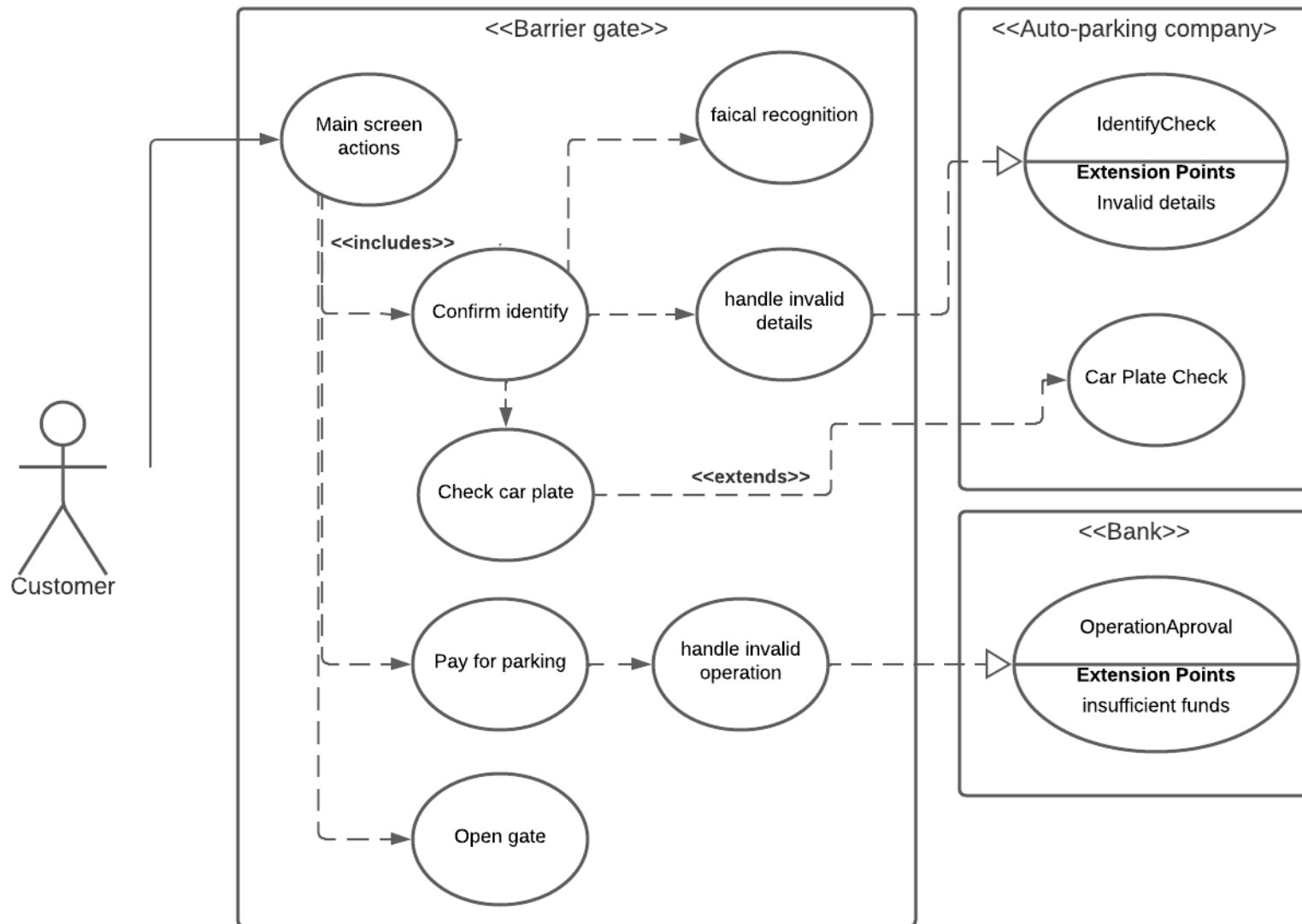
On arrival to an auto-barrier gate, approaching individual shall be facial recognized and the car plate shall be checked to know whether the the individual has a valid subscription and valid parking permit. This checking is done through the operation company of the auto-parking system.

- **Use case:** access the parking space

An individual wants to access the parking space and don't know if he/she still has a valid parking permit. If valid permit exists, then the gate shall open to the gate to the the customer to access. Alternatively, the auto-barrier gate shall provide the individual with a limited time parking permit that requires payment and only valid for sometime.

There are three external use cases involved: Identity check, car plate check, and transaction approval.

- **Failures:** occur during at all external use cases.



Requirements discovery (collection) (cont.d)

4. Ethnography: a complementary study of social and organizational context of operational processes to reveal critical details missed by other discovery techniques

- The analysis can be used to derive support and missing requirements
- Requirement analyst gets deeply into the working environment
- This helps to discover implicit requirements on actual ways the processes (define statically by clients) interact with people work (more dynamic).

Requirements validation

Checking that requirements actually define the system that the customer really wants. There are different types of checks:

1. **Validity:** systems have many stakeholders with different needs and requirements can be compromised across
2. **Consistency:** resolve conflict requirements
3. **Completeness:** requirements document must include requirements of all functions and constraints. No missing requirements.
4. **Realism:** ensure that requirements can actually be implemented
5. **Verifiability:** requirements should be written so that they are verifiable
(allow a set of tests to be written)

Requirements validation (cont.d)

There are a number of requirements validation techniques:

1. Requirements reviews

- checking requirements systematically for errors and inconsistencies

1. Prototyping

- Executable temporary version of the target system to end-users and customer

1. Test-case generation

- Testing requirements: create a test for requirement and find error. If test is difficult to do, this mean that the requirement is hard to test.
- In agile programming, engineers craft tests from user requirements before even coding.